5G-based IoT Gateway with Edge Machine Learning

Group 20
Arshman Tariq (2509523)
Juuso Antero Alatalo (Y6909125)
Muhammad Faizan Tanveer (2509524)
Seyedehsahar Fatemi Abhari (2508265)

***Abstract*—The rapid growth of Internet-of-Things (IoT) devices requires scalable and intelligent infrastructures capable of processing large volumes of sensor data with minimal latency. This project presents a distributed 5G-enabled IoT gateway architecture that integrates edge computing, cloud analytics, and machine learning for real-time anomaly detection. The system simulates thousands of IoT devices generating environmental sensor data, i.e. Temperature and Pressure, and processes this data using a microservices-based distributed architecture.**

Sensor data is transmitted from simulated devices to edge gateways using the MQTT protocol. Each gateway performs real-time anomaly detection using a lightweight machine learning model, i.e. Isolation Forest, deployed at the edge to provide immediate inference locally, while a more computationally intensive model, like Random Forest, is periodically trained in the cloud using Apache Spark on historical sensor data stored in PostgreSQL. This enables an adaptive edge–cloud continuum.

The system is fully containerised using Docker and orchestrated through Docker Compose to simulate a distributed datacenter environment. Multiple gateway instances can be scaled to process incoming data streams while ensuring fault tolerance through local state queues and distributed task offloading. Redis was initially used for temporary state sharing before transitioning to a more distributed architecture with gateway coordination and consensus mechanisms.

A middleware layer provides administrative control including authentication, device provisioning, logging, and monitoring dashboards. Cloud analytics are implemented using Grafana connected to PostgreSQL, enabling real-time visualisation of sensor trends, anomalies, and gateway performance.

System evaluation demonstrates low-latency processing, strong message delivery reliability under partial failures, and robust throughput across multiple gateway configurations. The results highlight the effectiveness of distributed edge computing combined with cloud analytics for large-scale IoT environments.

The full implementation of the project is available at the following GitHub Repository:
https://github.com/FaizanTanwir/DS2026-Project-Group-20

***Index Terms*—IoT Gateway, Edge Computing, Distributed Systems, Apache Spark, Cloud Analytics.**

I. Problem Statement

The proliferation of IoT devices has resulted in unprecedented volumes of data generated at the network edge. Traditional cloud-centric architectures introduce latency, bandwidth overhead, and scalability challenges when processing this data. To address these limitations, modern distributed systems integrate edge computing and machine learning to process data closer to its source.

This project aims to design and implement a distributed 5G-enabled IoT gateway capable of processing sensor data streams from thousands of devices while performing real-time anomaly detection. The system leverages edge computing for low-latency inference and cloud computing for large-scale analytics and model training.

The key technical challenges addressed in this project include:

- Efficient communication between IoT devices, gateways, and cloud services.
- Scalable processing of sensor data streams.
- Fault tolerance and state replication under node failures.
- Integration of edge ML with cloud-based training.
- Visualisation and monitoring of system performance.

To address these challenges, the system adopts a microservices-based architecture with containerized components, enabling scalability, modularity, and distributed processing across edge and cloud environments.

II. System Design

The system architecture follows an edge–cloud continuum where sensor data is generated at the edge, processed by distributed gateways, and stored in the cloud for analytics and model training.

The main components of the system include:

- **IoT Device Simulator**: Generates synthetic sensor data from thousands of devices.
- **MQTT Broker**: Handles publish/subscribe communication between devices and gateways.
- **Edge Gateways**: Perform real-time anomaly detection and forward results to the cloud.
- **Cloud Service**: Stores sensor data and performs analytics and distributed ML training.
- **Apache Spark Cluster**: Executes batch ML training on historical data.
- **PostgreSQL Database**: Provides persistent cloud storage.
- **Middleware Layer**: Manages device provisioning, authentication, and logging.
- **Grafana Dashboard**: Visualizes system analytics and metrics.

Figure 1 illustrates the overall workflow of the system.

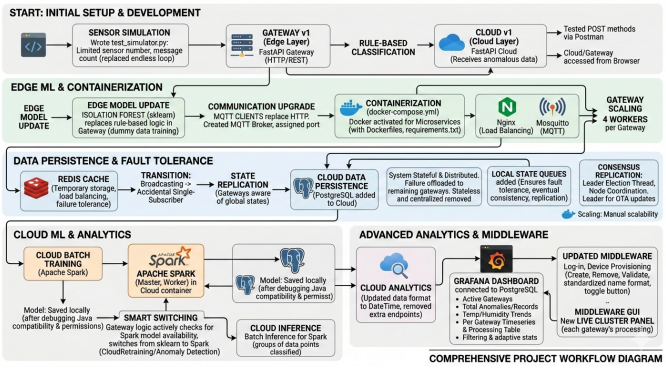

Fig. 1. Distributed IoT Gateway Architecture

The workflow begins with IoT devices publishing sensor data through MQTT. The edge gateways subscribe to these messages and process them using machine learning models to detect anomalies. Processed data is then forwarded to the cloud service, which stores the data in PostgreSQL and performs analytics using Grafana and model retraining using Apache Spark. The trained models are then deployed back to the edge.

III. Implementation

The project was developed iteratively through several increments, gradually evolving into a distributed microservices system.

Initially, a Python-based sensor simulator was implemented to generate environmental data such as temperature and humidity. Early versions used an infinite loop for continuous data generation, which was later replaced by controlled simulations with configurable device counts and message limits.

The first gateway implementation used FastAPI and HTTP communication for device-to-gateway interactions. Sensor data was classified using simple rule-based anomaly detection logic. However, this approach lacked scalability and practicality.

To improve communication efficiency, MQTT was introduced as the messaging protocol using the Eclipse Mosquitto broker. This enabled a publish–subscribe architecture suitable for large-scale IoT deployments.

Edge intelligence was introduced by integrating a machine learning model based on Isolation Forest using the `scikit-learn` library. This model performs real-time anomaly detection directly on the gateway.

The system was then containerised using Docker, and a Docker Compose environment was created to orchestrate all microservices. Components such as the MQTT broker, Nginx load balancer, Redis cache, PostgreSQL database, and FastAPI services were deployed as independent containers.

Redis was initially used to support temporary data replication and gateway coordination. Later, the architecture evolved into a more distributed system where gateways handled failures through task offloading and local state queues. This ensured eventual consistency and improved fault tolerance.

A middleware layer was also developed to provide device management, authentication, and monitoring capabilities. Leader election mechanisms were implemented among gateways to support coordination tasks such as over-the-air updates.

For cloud analytics and large-scale model training, Apache Spark was deployed using a master–worker architecture. Historical sensor data stored in PostgreSQL was used to train more advanced anomaly detection models in batch mode. The trained models were saved and made available to edge gateways.

A smart inference mechanism was implemented at the gateway layer. If a Spark-trained model is available, the system uses it for predictions; otherwise, it falls back to the lightweight edge model.

Finally, Grafana dashboards were integrated with PostgreSQL to provide real-time visualisation of sensor data trends, anomaly counts, and gateway processing metrics.

IV. Results

System evaluation was performed using multiple performance metrics, including latency, message delivery rate, and throughput.

A. Latency

Latency was measured as the difference between the cloud timestamp and the edge timestamp of sensor messages.

Two configurations were evaluated:

- Low workload: 10 sensors generating 50 messages
- High workload: 100 sensors generating 500 messages each (50,000 messages total)

The high workload simulation stopped data generation at **11:18:32.19**, while the final message was recorded in the cloud database at **11:18:32.66**. This corresponds to an approximate latency of:

$$Latency \approx 0.47 \ seconds$$

For the low workload configuration, data generation stopped at **14:46:39.39** and the last message reached the cloud at **14:46:39.43**, indicating extremely low latency.

These results demonstrate that the system maintains consistent processing performance even with increased message volumes.

Fig. 2. Latency Measurement

B. Message Delivery Rate

Message delivery rate was calculated as:

$$Delivery \ Rate = \frac{Messages \ Stored}{Messages \ Sent} \times 100$$

Two failure scenarios were tested.

Cloud Failure Scenario

During temporary cloud downtime, gateways buffered messages locally. Once the cloud recovered, all messages were delivered successfully.

$$Delivery\ Rate = \frac{5000}{5000} \times 100 = 100\%$$

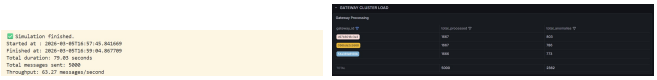

Fig. 3. Cloud Failure

Gateway Failure Scenario

When one gateway failed during operation, the remaining gateways processed most of the data.

$$Delivery\ Rate = \frac{4462}{5000} \times 100 \approx 89.2\%$$

This demonstrates partial fault tolerance and load redistribution across gateways.

Fig. 4. Cloud Failure

C. Throughput

Throughput was calculated using:

$$Throughput = \frac{Total\ Messages}{Total\ Time}$$

Two configurations were tested.

- Single Gateway: 73.09 messages/sec
- Five Gateways: 68.97 messages/sec

The results indicate stable system throughput even when scaling the gateway cluster.

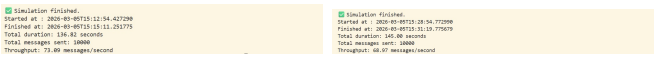

Fig. 5. System Throughput

D. System Outputs

System functionality was verified through outputs including:

- Docker container logs
- MQTT message logs
- Apache Spark training logs
- Gateway processing logs
- PostgreSQL database records
- Grafana analytics dashboards
- Middleware management dashboard

Example outputs are shown in Figures 6, 7, 8, 9, 10 and 11.

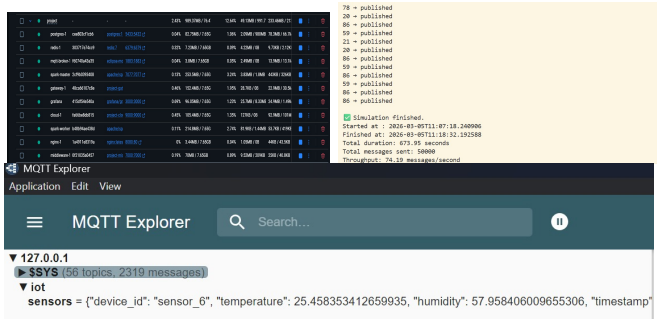

Fig. 6. Docker Containers, MQTT logs and Device Simulation

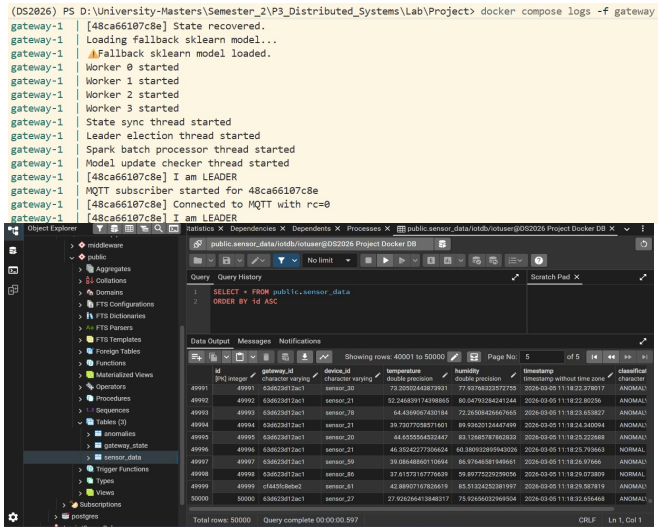

Fig. 7. Gateway Logs and PostgreSQL

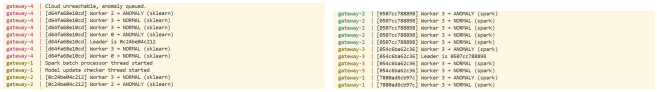

Fig. 8. Edge and Cloud ML Inferences

Fig. 9. Apache Spark and Batch Training

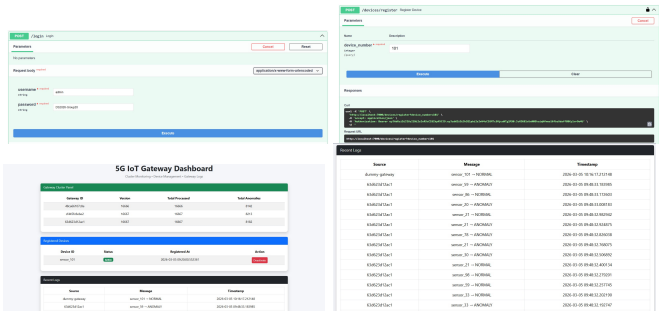

Fig. 10. Middleware Admin Roles and Dashboard

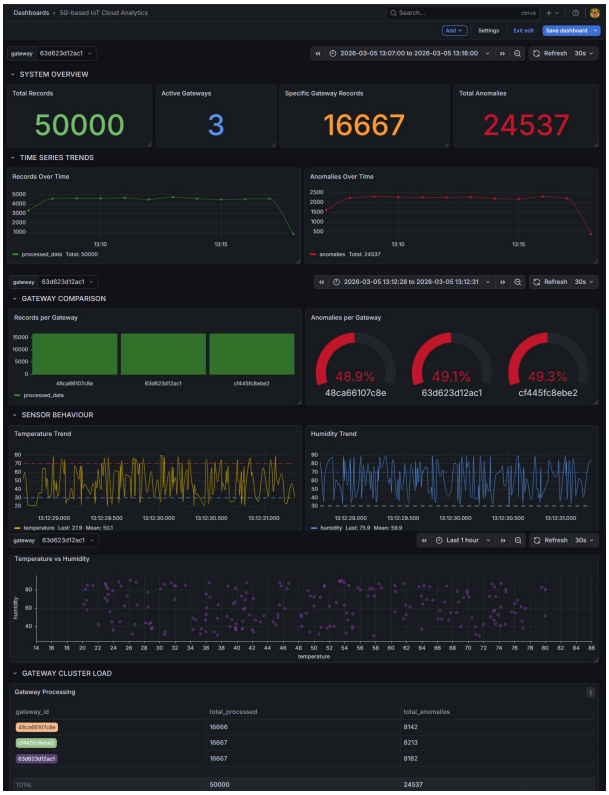

Fig. 11. Grafana Cloud Analytics Dashboard

V. Conclusion

This project demonstrates the design and implementation of a distributed IoT gateway architecture capable of processing large-scale sensor data streams using edge computing and cloud analytics. By combining MQTT communication, containerised microservices, distributed machine learning, and real-time monitoring dashboards, the system achieves scalable and resilient data processing.

Edge inference using lightweight models enables low-latency anomaly detection, while cloud-based training with Apache Spark allows continuous model improvement using historical data. The architecture supports manual scaling of gateway instances and provides fault tolerance mechanisms such as message buffering and distributed processing.

The results indicate strong system performance in terms of latency, throughput, and reliability under failure scenarios. Future improvements could include automated gateway scaling, adaptive model selection based on workload characteristics, and more advanced distributed consensus mechanisms.

Role of AI tools in the project

AI tools such as ChatGPT were used primarily for debugging assistance, architectural discussions, and resolving implementation issues during the development process. Also, we used ChatGPT to paraphrase and structure our report.

Division of Labour and Time Spent

Team Member	Time Spent	Work Done
Arshman Tariq	20 hrs	Eventual consistency, Worker Scalability, Cloud Fault Tolerance, PostgreSQL storage, Cloud Analytics using Grafana, and Report writing.
Juuso Antero Alatalo	20 hrs	Edge–gateway communication, Gateway Fault Tolerance, Load balancing, Containerization, and Report writing.
Muhammad Faizan Tanveer	20 hrs	Middleware development, Edge–cloud ML Switching, Batch Inference Using Apache Spark, Cloud Analytics using Grafana and Report Writing.
Seyedehsahar Fatemi Abhari	20 hrs	Edge ML Inference, Consensus and State replication, Middleware development, System Evaluation, and Report Writing.

Learning Dairy

This project provided valuable experience in designing and implementing a complex distributed systems.

One of the most important learning outcomes was understanding how microservice architectures can be built using Docker containers. Containerization allowed independent services such as gateways, cloud services, and analytics tools to be deployed and managed efficiently.

Working with Apache Spark provided insights into distributed data processing and batch machine learning training. Spark enabled scalable processing of historical sensor datasets for model training in a cloud environment.

The integration of Grafana with PostgreSQL demonstrated how data can be transformed into meaningful visual analytics. Real-time dashboards provided valuable insights into system performance, gateway activity, and anomaly trends.

Additionally, the project deepened understanding of core distributed systems concepts, including load balancing, replication, consensus mechanisms, fault tolerance, and the edge–cloud computing continuum.

Overall, the project was both challenging and rewarding, requiring integration of multiple technologies, including Python, Docker, FastAPI, MQTT, PostgreSQL, Apache Spark, HTML-based front-ends, and machine learning frameworks.

References

The full implementation of the project is available at the following GitHub Repository:

https://github.com/FaizanTanwir/DS2026-Project-Group-20